Characterizing Feature Sets for Just-In-Time Defect Prediction: A Systematic Literature Review

Firas Jolha
f.jolha@innopolis.university
Innopolis University
Innopolis, Russia

Adel Azzi
a.azzi@innopolis.university
Innopolis University
Innopolis, Russia

Demid Davletzianov
d.davletzianov@innopolis.university
Innopolis University
Innopolis, Russia

Artem Lisitcin
a.lisitcin@innopolis.university
Innopolis University
Innopolis, Russia

Giancarlo Succi
g.succi@unibo.it
University of Bologna – Alma Mater Studiorum
Bologna, Italy
Constructor Knowledge Labs and Constructor University
Bremen, Germany

Abstract

Just-In-Time (JIT) defect prediction flags software changes that are likely to introduce defects at the moment they are submitted. Every JIT model depends on the feature set used to describe a change, and the literature proposes many such features, from change-level metrics to semantic representations learned from code and commit messages. These features are named, defined and extracted in different ways, and the evidence on their usefulness is scattered across studies. Existing secondary studies cover JIT defect prediction as a whole and treat features as one aspect among several. We present the protocol and study selection of a Systematic Literature Review (SLR) focused on the feature sets themselves. Five research questions address their types and prevalence, their properties and extraction, their effect on predictive performance, their behavior under concept drift, and their use in within-project, cross-project, online and effort-aware settings. A search of six digital libraries returned 37,566 records, from which 212 primary studies published between 2006 and 2025 were selected.

CCS Concepts

• **Software and its engineering** → **Software defect analysis**; • **Computing methodologies** → *Machine learning*; • **General and reference** → *Surveys and overviews.*

Keywords

Just-In-Time defect prediction, feature sets, software metrics, concept drift, systematic literature review

1 Introduction

Software defects increase maintenance costs and reduce system reliability [3, 24]. Modern software evolves through small, incremental changes that are often integrated several times a day [9, 22]. Traditional defect prediction estimates the defect-proneness of modules, files, or classes at release time [7, 21]. Its predictions come too late, are too coarse-grained, and do not tell developers which change to inspect [14].

Just-In-Time (JIT) defect prediction works at the level of individual changes (commits). As soon as a change is submitted, the model assesses whether it is likely to be defect-inducing [14, 15, 18]. At this point the code to inspect is small, the author still remembers the design decisions, and it is clear who is responsible for the change [14].

Every JIT model relies on how a change is represented, that is, on its feature set. Early studies used hand-crafted change-level metrics for the size, diffusion, purpose and history of a change and the experience of its developers [14]. Later work added features from commit messages, code review, static analysis, and the social and process context of development [2, 6]. Deep learning methods now learn semantic representations directly from the changed code and the commit message [10–12, 26], although simple change metrics remain highly competitive [27].

This variety leads to three problems. First, there is no shared understanding of which feature sets are used and how they are defined and extracted. The same features appear under different names, and extraction methods are rarely reported in full. Second, the evidence on which features improve predictive performance is spread across studies with different datasets, models and evaluation criteria. Third, the value of a feature changes over time. As projects evolve, feature distributions and their link to defects shift (concept drift) [17]. Usefulness also depends on the setting, which can be within-project or cross-project [5, 13], offline or online [4, 16], and effort-aware, where changes are ranked by predicted defect-proneness per unit of inspection effort [14, 25].

Existing secondary studies of JIT defect prediction [1, 28] treat features as one aspect among several. To the best of our knowledge, no systematic review has focused on the feature sets themselves. We therefore conduct a Systematic Literature Review (SLR) guided by five research questions:

RQ1 What kinds of feature sets are used in JIT defect prediction, which ones are used most often, and how has their use changed over time?

RQ2 How are these feature sets characterized in terms of their properties and the methods used to extract them?

RQ3 What effect do different feature sets have on the predictive performance of JIT defect prediction models?

RQ4 How does the dynamic behavior of features (concept drift) affect their usefulness in JIT defect prediction?

RQ5 How are the feature sets employed in JIT defect prediction used in practice, and what challenges and limitations arise in different settings?

This paper contributes (i) a review protocol for studying feature sets in JIT defect prediction, with a quality assessment tailored to feature-related reporting, (ii) the outcome of study selection, with 212 primary studies selected from 37,566 records, and (iii) a data extraction and synthesis plan that maps each extracted item to RQ1–RQ5, covering within-project, cross-project, online and effort-aware settings.

2 Related Work

Most secondary studies on defect prediction focus on the module, file, or class level. Hall et al. [7] analyzed how modeling approaches affect performance. Radjenović et al. [21] found that object-oriented and process metrics outperform size and complexity metrics. Pal and Sillitti [19] reviewed cross-project defect prediction, including feature selection. Qiu et al. [20] found a shift from traditional metrics to semantic and network metrics. None of them focuses on individual changes or on the temporal behavior of features. Only a few reviews target JIT defect prediction. Zhao et al. [28] reviewed 67 JIT studies and meta-analyzed their reported performance. Alnagi and Azzeh [1] reviewed 30 studies from 2012–2023 on techniques, datasets and metrics. Both cover the field as a whole. Table 1 compares these studies with this review.

2.1 Background on JIT Defect Prediction

Training data for JIT models is usually obtained by tracing bug-fixing commits back to the changes that introduced the fixed lines, most often with the SZZ algorithm [23]. Each change is then represented by a feature set, and a classifier or ranking model is trained on this labeled history. Kamei et al. [14] proposed 14 change-level metrics in five dimensions, namely diffusion, size, purpose, history and developer experience, which remain the most common baseline. The added value of richer representations is still debated. Zeng et al. [27] showed that a model using only the number of added lines can match deep JIT models. Herbold et al. [8] found that only about half of the bug-fixing commits identified by SZZ actually fix bugs, a more severe threat to validity than small feature sets. How features and labels are defined and extracted can therefore matter as much as the learning algorithm.

3 Methodology

3.1 Literature Search

We searched six digital libraries between September 2025 and October 2026. The search returned 37,566 records from Google Scholar (18,901), SpringerLink (10,065), Scopus (4,999), ACM Digital Library (3,062), ScienceDirect (478) and IEEE Xplore (61). The same query was applied to all sources:

```
(("buggy" OR "bug inducing" OR "anomalous" OR
"just in time defect" OR "just in time bug" OR
"JIT defect" OR "JIT bug") AND ("fault" OR "defect")
AND ("commit" OR "change")) AND
```

Table 1: Comparison with related secondary studies.

Study	Studies	Feat. focus	Extr. methods	Perf. impact	Concept drift	Settings
Module-level						
[7]	208	(✓)	–	(✓)	–	–
[21]	106	✓	–	✓	–	–
[19]	[n]	(✓)	–	(✓)	–	CP
[20]	[n]	✓	✓	(✓)	–	[verify]
Change-level (JIT)						
[28]	67	(✓)	(✓)	(✓)	(✓)	CP, OL, EA
[1]	30	–	–	–	–	CP, EA
Ours	**212**	✓	✓	✓	✓	**WP, CP, OL, EA**

✓: central research question; (✓): discussed, not a primary focus; –: not addressed. WP = within-project, CP = cross-project, OL = online, EA = effort-aware.

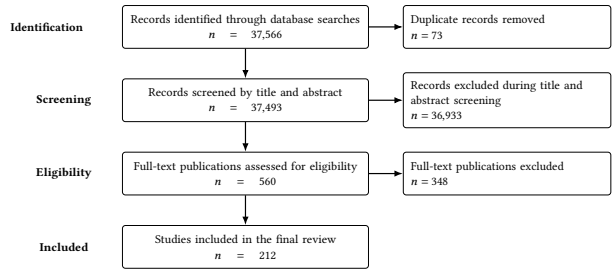


Figure 1: PRISMA diagram of the study selection process

```
("detection" OR "classification" OR "prediction") AND
("feature set" OR "features" OR "feature selection" OR
"feature engineering" OR "feature importance" OR
"feature characterization" OR "metrics")
```

3.2 Inclusion and Exclusion Criteria

We included peer-reviewed journal and conference papers in English, with accessible full text, published within a 20-year window, that address defect prediction at the commit or change level and explicitly discuss the features, feature sets, or metrics used (IC1–IC5). We excluded studies that are not peer-reviewed, duplicate other reports, fall outside software defect prediction, report no empirical results, or address only module-, file-, or class-level prediction (EC1–EC6). We also excluded studies that do not use features to build a JIT model, describe their datasets or feature extraction insufficiently, lack bibliographic information, are not primary studies, have no accessible full text, or fail the quality assessment (EC7–EC12).

3.3 Study Selection

After removing 73 duplicates, we screened 37,493 records by title and abstract and excluded 36,933. We assessed the full text of the remaining 560 publications against the criteria and excluded 348. The screening work was divided among the team, and every decision was recorded in a shared spreadsheet. The process resulted in 212 primary studies published between 2006 and 2025 (Figure 1).

3.4 Quality Assessment

Each study is assessed with the twelve questions (QA1–QA12) in Table 2, answered as *Yes* (1), *Maybe* (0.5) or *No* (0). QA1–QA3 ask whether the study uses several types of change characteristics, several feature sets, and semantic features. QA4–QA7 ask whether

Table 2: Quality assessment questions and scoring criteria

ID	Question	Yes (1)	Maybe (0.5)	No (0)
QA1	Does the study use features or metrics that leverage various characteristics of code changes for JIT defect prediction?	Uses features that capture at least two types of code-change characteristics (e.g., change size and diffusion).	Uses features that capture only one type of code-change characteristics (e.g., change size).	Uses no features that capture code-change characteristics (e.g., only developer-related features).
QA2	Does the study identify different sets of features used in JIT defect prediction?	Uses at least two different feature sets (e.g., AST-based features and the expert features of Kamei et al.).	Relies on a single feature set (e.g., the expert features of Kamei et al.).	Does not specify any feature set.
QA3	Does the study use semantic features that represent the underlying semantics of code changes?	Uses both expert and semantic features.	Uses only semantic features.	Uses only expert features.
QA4	Does the study sufficiently describe the datasets used in the experiments?	Provides sufficient details about the datasets used.	Uses some datasets without enough detail about their description.	Lacks a description of the datasets used.
QA5	Does the study report the methods and tools used for feature extraction and selection?	Clearly defines the methods and tools used for extracting and selecting features.	Uses external tools for extracting features.	Lacks the methodology and tools for extracting and selecting features.
QA6	Does the study validate the methods or tools used for feature extraction and selection?	Empirically validates the methods and tools used for feature extraction and selection.	Provides a validation scheme without any empirical work.	Does not validate the methods and tools used for feature extraction and selection.
QA7	Does the study evaluate the impact of different feature combinations on JIT defect prediction?	Examines the effect of different feature sets on JIT defect prediction performance.	Measures the effect of a single feature set on JIT defect prediction performance.	Reports performance for one feature set without characterizing any feature subsets.
QA8	Does the study investigate online JIT defect prediction?	Reports the performance of JIT defect prediction in an online setting.	Discusses online JIT defect prediction without relevant experiments.	Experiments only with a static set of features and does not discuss online prediction.
QA9	Does the study incorporate methods for detecting concept drift in online JIT defect prediction?	Examines several methods for detecting concept drift.	Uses a single method for detecting concept drift.	Does not discuss concept drift.
QA10	Does the study measure the performance of JIT defect prediction models in the presence of concept drift?	Reports changes in model performance in the presence of concept drift.	Provides some information on performance changes under concept drift, but not based on empirical work.	Provides no information on performance changes under concept drift.
QA11	Does the study investigate effort-aware JIT defect prediction?	Uses metrics to measure the performance of effort-aware JIT defect prediction.	Discusses such metrics but does not use them.	Does not mention metrics for effort-aware JIT defect prediction.
QA12	Does the study conduct experiments on cross-project JIT defect prediction?	Provides empirical results for cross-project JIT defect prediction.	Discusses cross-project performance without empirical investigation.	Does not discuss cross-project JIT defect prediction.

it describes its datasets, reports and validates its feature extraction and selection methods, and evaluates feature combinations. QA8–QA12 ask whether it investigates online prediction, detects concept drift, measures performance under drift, and studies effort-aware and cross-project prediction. The overall score ranges from 0 to 12, and studies scoring below [3] were excluded under EC12. The threshold is low on purpose, because QA8–QA12 cover specialized settings that many relevant studies do not address.

3.5 Data Extraction and Synthesis

A structured form extends the screening spreadsheet with five groups of items. These are feature characteristics (names, descriptions, categories, formulas) and feature sets with their extraction methods and tools (RQ1, RQ2), performance metrics, reported effectiveness and contribution of feature sets, models and projects (RQ3), whether and how concept drift is detected and its effect on performance (RQ4), and model performance in within-project and cross-project settings (RQ5). We synthesize the data with descriptive summaries and qualitative analysis. The review runs no new experiments. Findings are interpreted in the context of each study's setting, and numerical comparisons are made only when methodologically justified.

4 Current Status and Next Steps

We presented the protocol of an SLR on feature sets for JIT defect prediction and the outcome of its study selection, with 212 primary studies selected from 37,566 records. The next step is to extract and synthesize data from these studies to answer RQ1–RQ5. The expected outcome is a classification of JIT feature sets, a description of how they are extracted, a summary of the evidence on their predictive value, and an account of how studies handle concept drift and different prediction settings.

References

[1] Eman Alnagi and Mohammad Azzeh. 2024. Just-in-Time Software Defect Prediction Techniques: A Survey. In *Proceedings of the 15th International Conference on Information and Communication Systems (ICICS)*. 1–6. doi:10.1109/ICICS63486.2024.10638276

[2] Amir T. Barzoki and Kostas Kontogiannis. 2025. Predicting Bug Inducing Commits Using Commit-State Transition Analysis. In *Proceedings of the 49th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*. doi:10.1109/COMPSAC65507.2025.00165

[3] Barry Boehm and Victor R. Basili. 2001. Software Defect Reduction Top 10 List. *Computer* 34, 1 (2001), 135–137. doi:10.1109/2.962984

[4] George G. Cabral, Leandro L. Minku, Emad Shihab, and Suhaib Mujahid. 2019. Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. 666–676. doi:10.1109/ICSE.2019.00076

[5] Hongming Dai, Jianqing Xi, and Hong-Liang Dai. 2025. Improve Cross-Project Just-in-Time Defect Prediction with Dynamic Transfer Learning. *Journal of Systems and Software* 219 (2025), 112214. doi:10.1016/j.jss.2024.112214

[6] Emanuela Guglielmi, Andrea D'Aguanno, Rocco Oliveto, and Simone Scalabrino. 2025. Enhancing Just-In-Time Defect Prediction Models with Developer-Centric Features. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 52–62. doi:10.1109/MSR66628.2025.00021

[7] Tracy Hall, Sarah Beecham, David Bowes, David Gray, and Steve Counsell. 2012. A Systematic Literature Review on Fault Prediction Performance in Software Engineering. *IEEE Transactions on Software Engineering* 38, 6 (2012), 1276–1304. doi:10.1109/TSE.2011.103

[8] Steffen Herbold, Alexander Trautsch, Fabian Trautsch, and Benjamin Ledel. 2022. Problems with SZZ and Features: An Empirical Study of the State of Practice of Defect Prediction Data Collection. *Empirical Software Engineering* 27, 2 (2022), 42. doi:10.1007/s10664-021-10092-4

[9] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 426–437. doi:10.1145/2970276.2970358

[10] Thong Hoang, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction. In *Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 34–45. doi:10.1109/MSR.2019.00016

[11] Thong Hoang, Hong Jin Kang, David Lo, and Julia Lawall. 2020. CC2Vec: Distributed Representations of Code Changes. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 518–529. doi:10.1145/3377811.3380361

[12] Yuze Jiang, Beijun Shen, and Xiaodong Gu. 2025. Just-in-Time Software Defect Prediction via Bi-modal Change Representation Learning. *Journal of Systems and Software* 219 (2025), 112253. doi:10.1016/j.jss.2024.112253

[13] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. 2016. Studying Just-in-Time Defect Prediction Using Cross-Project Models. *Empirical Software Engineering* 21, 5 (2016), 2072–2106. doi:10.1007/s10664-015-9400-x

[14] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering* 39, 6 (2013), 757–773. doi:10.1109/TSE.2012.70

[15] Sunghun Kim, E. James Whitehead, Jr., and Yi Zhang. 2008. Classifying Software Changes: Clean or Buggy? *IEEE Transactions on Software Engineering* 34, 2 (2008), 181–196. doi:10.1109/TSE.2007.70773

[16] Xutong Liu, Yufei Zhou, Yutian Tang, Junyan Qian, and Yuming Zhou. 2025. Human-in-the-loop Online Just-in-time Software Defect Prediction: What Have We Achieved and What Do We Still Miss? *Science of Computer Programming* 244 (2025), 103296. doi:10.1016/j.scico.2025.103296

[17] Shane McIntosh and Yasutaka Kamei. 2018. Are Fix-Inducing Changes a Moving Target? A Longitudinal Case Study of Just-in-Time Defect Prediction. *IEEE Transactions on Software Engineering* 44, 5 (2018), 412–428. doi:10.1109/TSE.2017.2693980

[18] Audris Mockus and David M. Weiss. 2000. Predicting Risk of Software Changes. *Bell Labs Technical Journal* 5, 2 (2000), 169–180. doi:10.1002/bltj.2229

[19] Sourabh Pal and Alberto Sillitti. 2022. Cross-Project Defect Prediction: A Literature Review. *IEEE Access* 10 (2022), 118697–118717. doi:10.1109/ACCESS.2022.3221184

[20] Shaoming Qiu, Bicong E, Jingjie He, and Liangyu Liu. 2025. Survey of Software Defect Prediction Features. *Neural Computing and Applications* 37, 4 (2025), 2113–2144. doi:10.1007/s00521-024-10937-1

[21] Danijel Radjenović, Marjan Heričko, Richard Torkar, and Aleš Živkovič. 2013. Software Fault Prediction Metrics: A Systematic Literature Review. *Information and Software Technology* 55, 8 (2013), 1397–1418. doi:10.1016/j.infsof.2013.02.009

[22] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2017. Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access* 5 (2017), 3909–3943. doi:10.1109/ACCESS.2017.2685629

[23] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When Do Changes Induce Fixes?. In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR)*. 1–5. doi:10.1145/1083142.1083147

[24] Gregory Tassey. 2002. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. Planning Report 02-3. National Institute of Standards and Technology.

[25] Yibiao Yang, Yuming Zhou, Jinping Liu, Yangyang Zhao, Hongmin Lu, Lei Xu, Baowen Xu, and Hareton Leung. 2016. Effort-Aware Just-in-Time Defect Prediction: Simple Unsupervised Models Could Be Better than Supervised Models. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 157–168. doi:10.1145/2950290.2950353

[26] Yujie Ye, Huiqun Yu, Guisheng Fan, Yuguo Liang, Jianan Dong, and Wentao Chen. 2025. JIT-Align: A Semantic Alignment-Based Ranking Framework for Just-In-Time Defect Prediction. In *Proceedings of the 49th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*. 1328–1337. doi:10.1109/COMPSAC65507.2025.00166

[27] Zhengran Zeng, Yuqun Zhang, Haotian Zhang, and Lingming Zhang. 2021. Deep Just-in-Time Defect Prediction: How Far Are We?. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 427–438. doi:10.1145/3460319.3464819

[28] Yunhua Zhao, Kostadin Damevski, and Hui Chen. 2023. A Systematic Survey of Just-in-Time Software Defect Prediction. *Comput. Surveys* 55, 10 (2023), 1–35. doi:10.1145/3567550